COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 24 — Configuration Management (Git & Change Control)

Section 12: Classroom Practical — Git Version Control for CEMS

Complete hands-on terminal execution log, branching strategy, conflict resolution, release tagging, and live GitHub remote push evidence for the College Event Management System (CEMS). Repository live on GitHub at: https://github.com/Daksh-create349/CEMS-SEPM (Screenshots archived in Screenshots/ directory).

Step 1 & 2: Local Repository Initialization & Initial Commits

- **1. Initialize Repository:** Command executed: git init -b main. Configured local identity: Daksh Srivastava (dakshshrivastav56@gmail.com / 150096725087).
- **2. Initial Commit (Root):** Created README.md and initial src/event_controller.js. Executed: git add . && git commit -m "chore: initial repository structure and CEMS documentation" (SHA: e08ae7c).
- **3. Second Commit on Main:** Added student roll number verification function validateStudentEligibility(). Executed: git add src/event_controller.js && git commit -m "feat(events): add student eligibility verification [CEMS-101]" (SHA: bc9ac0b).

Step 3 & 4: Feature Branching & Development

- **1. Branch Creation:** Created isolated feature branch: git checkout -b feature/event-list.
- **2. Feature Implementation:** Added src/event_list.js (category filtering) and modified EventController constructor to accept category. Staged and committed: git commit -m "feat(events): add category filtering and update controller [CEMS-102]" (SHA: cbc4b2f).

Step 5 & 6: Simulating & Resolving Merge Conflict

- **1. Divergent Change on Main:** Switched back to main branch (git checkout main) and modified src/event_controller.js on the identical constructor lines to introduce a waitlist parameter: constructor(eventId, capacity, maxWaitlist = 20). Committed on main: git commit -m "fix(events): add waitlist support parameter to controller [CEMS-104]" (SHA: d8f1b8b).
- **2. Trigger Merge Conflict:** Executed merge command: git merge feature/event-list. Output: CONFLICT (content): Merge conflict in src/event_controller.js. Automatic merge failed; fix conflicts and then commit the result.
- **3. Conflict Inspection:** Git status reported both modified: src/event_controller.js. File contained conflict markers between HEAD (main) and feature/event-list.
- **4. Manual Conflict Resolution:** Edited src/event_controller.js to combine both changes into a single unified constructor: constructor(eventId, capacity, category = "General", maxWaitlist = 20).

Removed all conflict markers, staged with git add src/event_controller.js, and finalized merge commit (SHA: d34a81b).

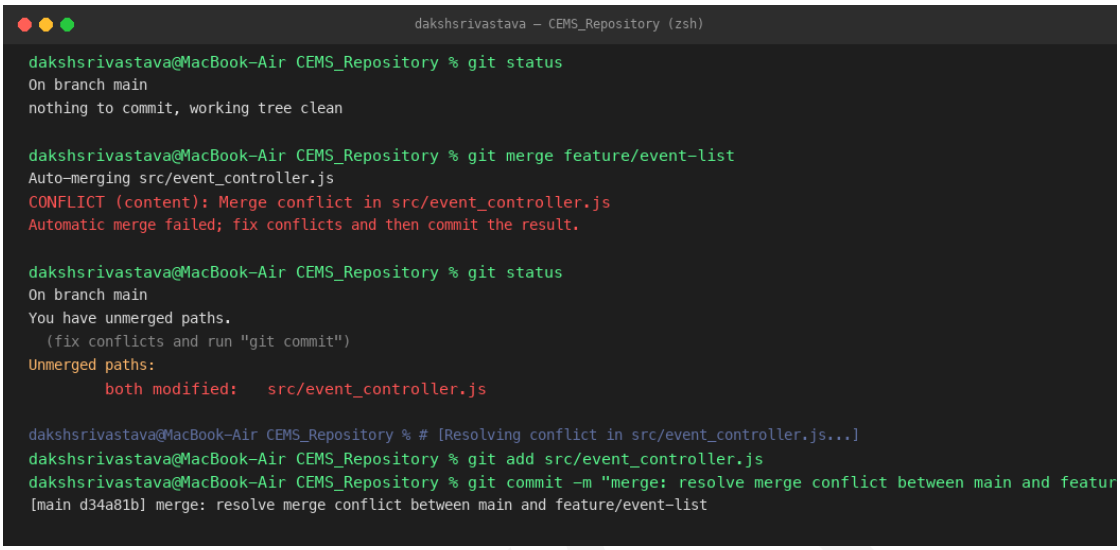


Figure 24.1: Terminal output demonstrating merge conflict simulation in src/event_controller.js, conflict markers, and successful manual resolution commit (d34a81b).

Step 7 & 8: Release Tagging & Live GitHub Remote Push

- **1. Annotated Release Tag:** Created cryptographic annotated tag: git tag -a v1.0.0 -m "Release v1.0.0: Initial stable CEMS release with event discovery and registration". Verified tag details via git tag -n.
- **2. Live GitHub Push:** Configured remote origin to git@github.com:Daksh-create349/CEMS-SEPM.git and pushed main branch and tags: git push -u origin main --tags. Also pushed feature branch: git push origin feature/event-list. Verified live at https://github.com/Daksh-create349/CEMS-SEPM.
- **3. Git Commit Graph Verification:** Verified complete commit history using git log --graph --oneline --all:

  * d34a81b (HEAD -> main, tag: v1.0.0, origin/main) merge: resolve merge conflict by integrating category and waitlist

  |\

  | * cbc4b2f (origin/feature/event-list, feature/event-list) feat(events): add category filtering and update controller [CEMS-102]

  * | d8f1b8b fix(events): add waitlist support parameter to controller [CEMS-104]

  |/

  * bc9ac0b feat(events): add student eligibility verification [CEMS-101]

  * e08ae7c chore: initial repository structure and CEMS documentation

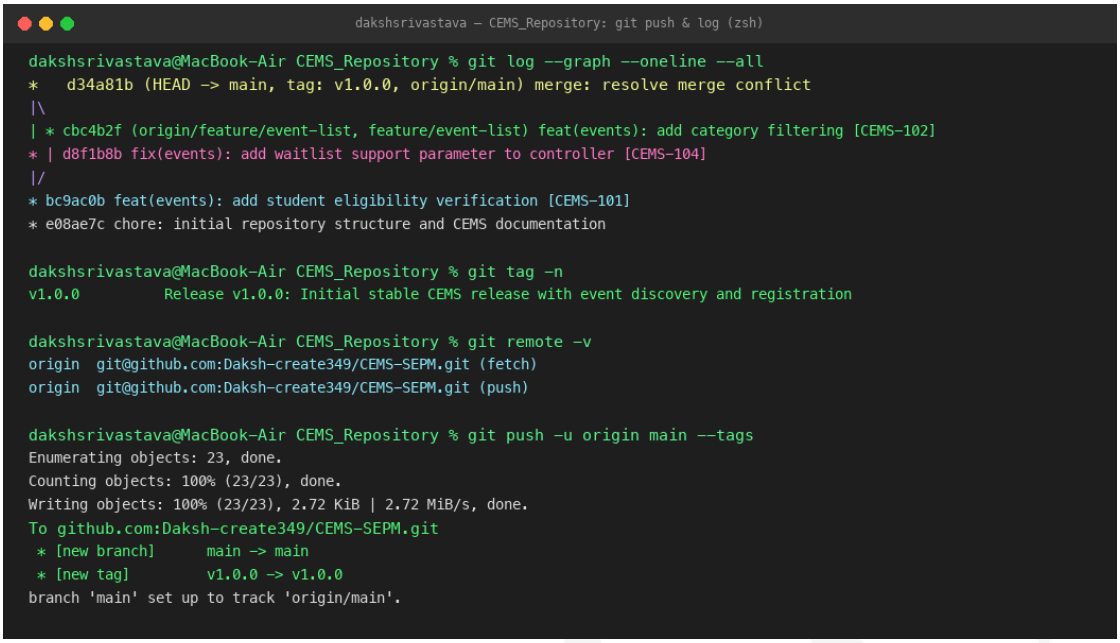


Figure 24.2: Terminal log displaying git log graph, annotated release tag v1.0.0, and push confirmation to GitHub remote.

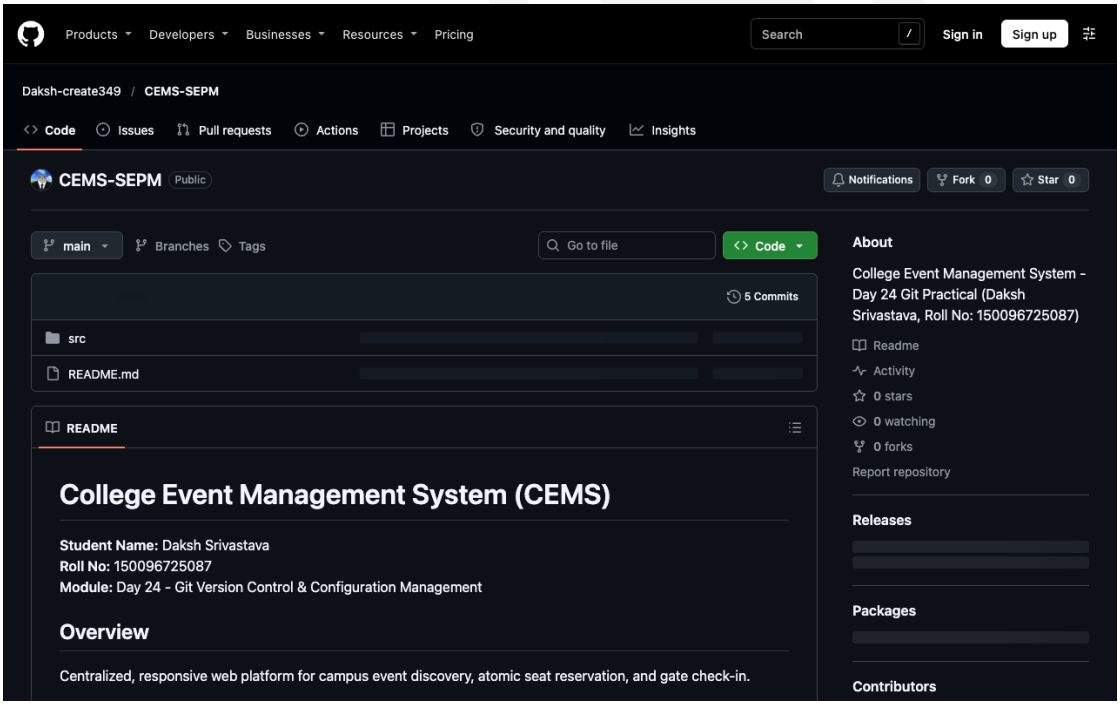


Figure 24.3: Live GitHub repository interface for CEMS (https://github.com/Daksh-create349/CEMS-SEPM) showing pushed commits, branches, and tag v1.0.0.

Section 16: Homework — Git for Library System & Change Management

Hands-on repository implementation for the Library Management System (Items 1–3) combined with enterprise change management and traceability procedures (Items 4–5). Repository live on GitHub at: https://github.com/Daksh-create349/LMS-SEPM.

Item 1 & 2: LMS Repository Initialization & Three Meaningful Commits

Commit Hash	Branch	Commit Message	Technical Scope & Justification
bd18744	main	docs: initialize LMS project charter and README [LMS-01]	Initializes project repository, defines project charter, architecture scope, and student metadata.
bab69e9	main	feat(catalog): implement ISBN indexing and catalog search module [LMS-02]	Adds src/catalog.py with dictionary-backed inventory indexing and constant-time ISBN title lookup.
50a2679	main	feat(circulation): implement 14-day loan policy and automated fine calculation [LMS-03]	Adds src/circulation.py enforcing 14-day circulation limits and automated Rs. 5/day fine arithmetic.

Item 3: Feature Branching, Diff Review & Merge

- **1. Feature Branch Creation:** Executed git checkout -b feature/borrower-renewal. Added src/renewal_service.py to enforce a maximum of 2 renewals and block renewal if an active reservation hold exists.
- **2. Pre-Merge Diff Review:** Executed git diff main..feature/borrower-renewal. Verified that 2 files were altered (+13 lines in src/renewal_service.py and documentation in README.md) with zero regressions to existing circulation rules.
- **3. Non-Fast-Forward Merge:** Switched to main and merged: git merge --no-ff feature/borrower-renewal -m "merge: integrate borrower renewal feature branch into main [LMS-04]" (Commit SHA: 5b3df83). Pushed to GitHub: git push -u origin main && git push origin feature/borrower-renewal.

DRAFT

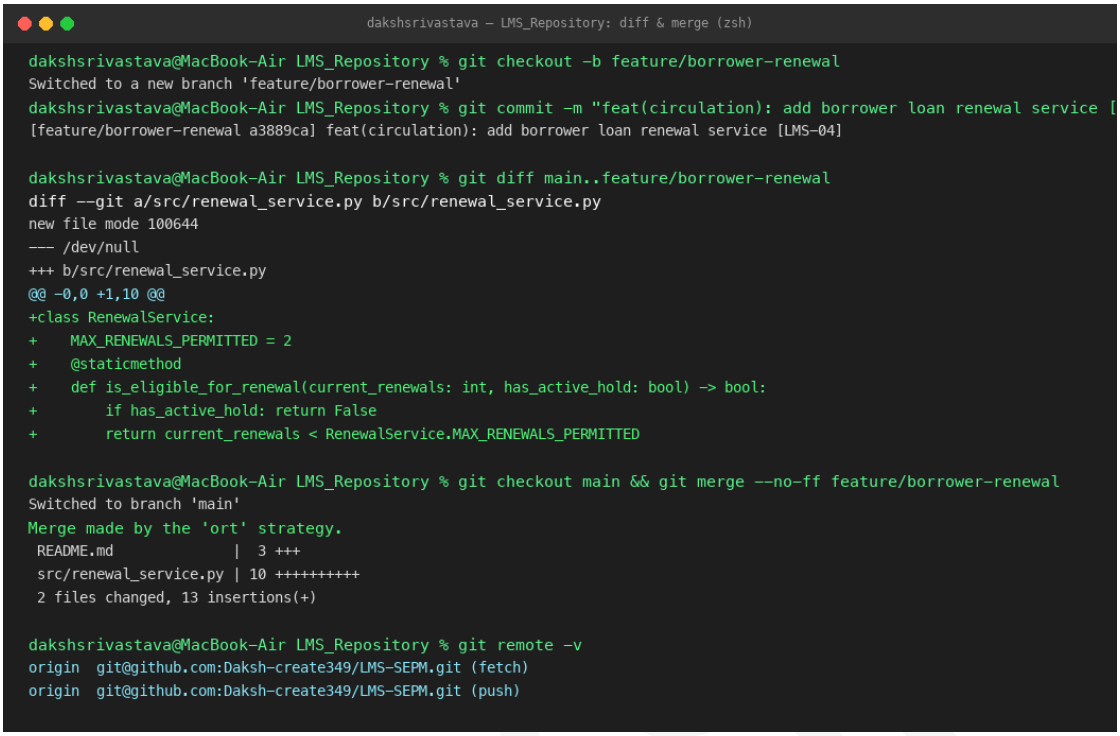


Figure 24.4: Terminal execution log for LMS repository feature branch creation, git diff inspection, non-fast-forward merge, and remote push.

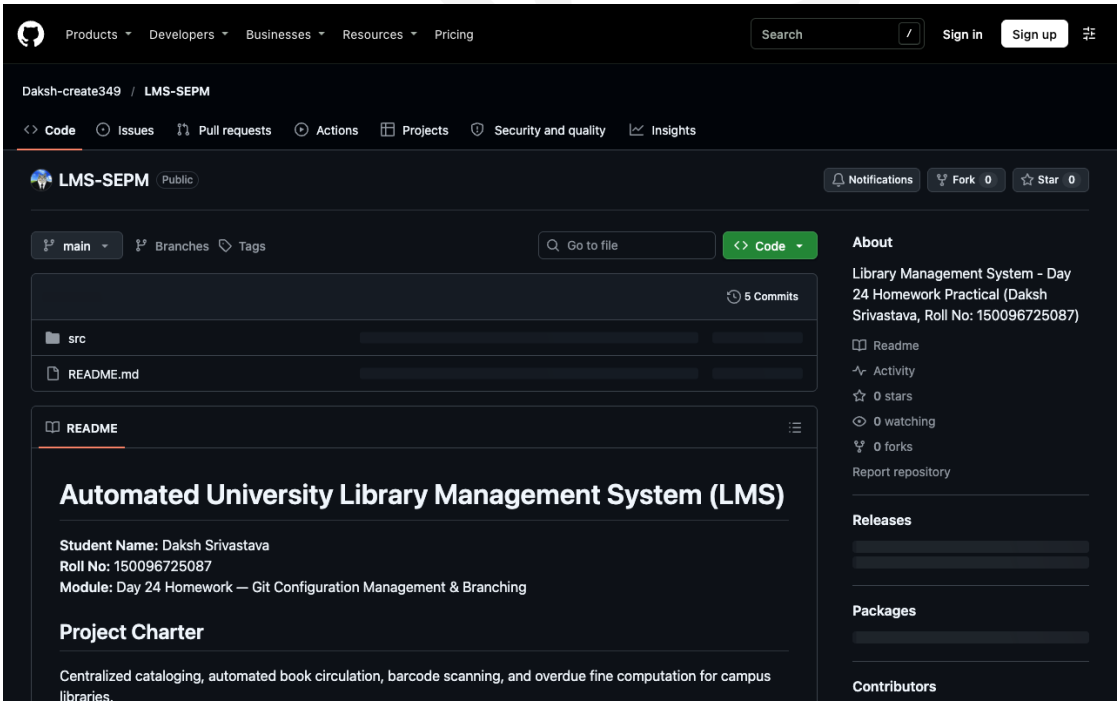


Figure 24.5: Live GitHub repository interface for LMS (https://github.com/Daksh-create349/LMS-SEPM) showing pushed commits, branches, and renewal service.

Item 4: Formal Change Request Specification (CR-CEMS-2026-08)

Field Name	Specification Details

Change Request ID	CR-CEMS-2026-08
Title / Summary	Introduce Dynamic Waitlist Queue & Automated Seat Escalation for Full Events
Originator & Role	Rohan Sharma, Head of Student Technical Festival Operations
Business Rationale	High-demand events (e.g. AI Workshop) fill within 5 minutes. 15% of registered students fail to attend. A dynamic waitlist reclaims unused seats and maximizes venue utilization.
Impacted Configuration Items	1. SRS Section 3.2 (Requirements Baseline v1.2) 2. Database: registrations table schema (add WAITLISTED status) 3. Backend API: POST /api/v1/registrations/waitlist 4. Frontend: Event Card UI badge & waitlist modal
Acceptance Criteria	1. System places 101st registrant into waitlist position #1. 2. On cancellation, position #1 is sent an offer email with 4-hour confirmation token. 3. If unconfirmed within 4h, seat cascades to position #2.
CCB Approval Decision	APPROVED by Change Control Board (Lead Architect, Project Manager, QA Lead)

Item 5: End-to-End Release Traceability Procedure

How to trace a production release artifact back to its origin commit, change request, and verification evidence:

1. Semantic Version Tagging: In Git, production releases are tagged with cryptographically signed annotations (e.g., git tag -a v1.2.0 -m "Release v1.2.0: Waitlist Feature"). The tag records the exact SHA-256 commit hash.

2. Commit Message Association: Every commit references its Jira ticket and CR number (e.g., git commit -m "feat(booking): implement FIFO waitlist queue [CR-08, CEMS-142]").

3. CI/CD Build Audit Trail: GitHub Actions / Jenkins logs compile the commit hash into the Docker container image metadata label (org.opencontainers.image.revision).

4. RTM Verification: The Requirements Traceability Matrix maps CR-08 -> Jira CEMS-142 -> Git Branch feature/waitlist -> Unit Test BookingWaitlistTest.java -> Test Run Report #4892. This establishes an unbroken chain of custody from business approval to running bytecode.